

Chương 2 đã trình bày cơ sở lý thuyết, khảo sát các hướng tiếp cận và xác định Retrieval-Augmented Generation là hướng tiếp cận phù hợp cho bài toán hỏi đáp trên SGK Tin học THPT. Trên cơ sở đó, chương này mô tả phương pháp được đề xuất, bao gồm kiến trúc tổng thể, các thành phần xử lý chính và cách tổ chức đánh giá cho các node trọng tâm trong pipeline.

Nội dung chương được trình bày theo hướng nghiên cứu, tập trung vào dữ liệu, retrieval, reranking, generation và evaluation, không đi sâu vào chi tiết cài đặt phần mềm.

0.1 Tổng quan kiến trúc hệ thống

0.1.1 Bài toán và yêu cầu

Bài toán của đề án là xây dựng một trợ lý giáo dục cho môn Tin học THPT, hỗ trợ người học và giáo viên khai thác nội dung từ sách giáo khoa trong môi trường tương tác số. Phạm vi dữ liệu của hệ thống được giới hạn vào sách giáo khoa Tin học lớp 10, 11 và 12 của hai bộ sách Cánh Diều và Kết Nối Tri Thức. Trên nền dữ liệu này, hệ thống không chỉ thực hiện tác vụ hỏi đáp kiến thức mà còn hỗ trợ các tác vụ sinh học liệu có cấu trúc, đòi hỏi khả năng tổng hợp và tổ chức lại nội dung.

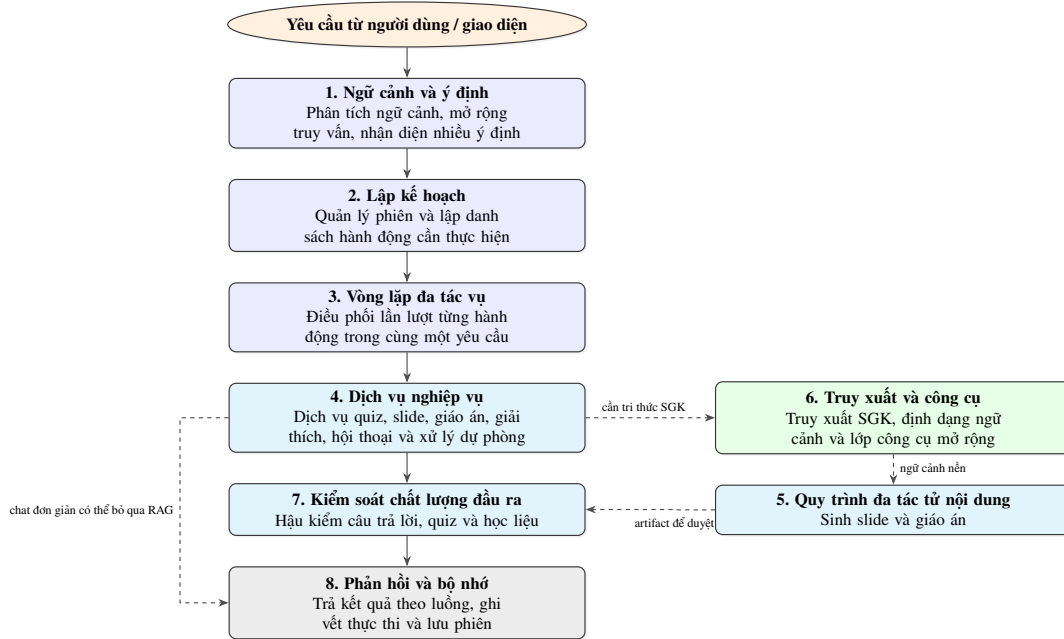
Ba nhóm tác vụ chính của hệ thống gồm hỏi đáp dựa trên SGK, sinh slide bài giảng và soạn giáo án. Với tác vụ hỏi đáp, hệ thống cần truy xuất các đoạn ngữ cảnh liên quan thông qua *Retriever*, sau đó sử dụng *LLM* để sinh câu trả lời bám sát tài liệu nguồn. Với tác vụ sinh slide và giáo án, hệ thống cần bao phủ nội dung bài học ở mức tổng thể, đồng thời tổ chức lại tri thức theo trình tự sư phạm thay vì chỉ trả về các đoạn văn bản rời rạc. Do đó, bài toán không dừng ở một pipeline *Retrieval-Augmented Generation* (RAG) đơn giản, mà mở rộng thành bài toán điều phối nhiều bước xử lý với các yêu cầu khác nhau về truy xuất, tổng hợp và kiểm soát chất lượng.

Trên cơ sở đó, phương pháp đề xuất cần thỏa mãn một số yêu cầu chính. Thứ nhất, hệ thống phải khai thác tri thức từ SGK theo đúng phạm vi lớp, bộ sách và chủ đề nhằm hạn chế sai lệch ngữ cảnh. Thứ hai, cơ chế truy xuất phải xử lý được cả câu hỏi cụ thể lẫn các yêu cầu tổng hợp nội dung ở mức bài học. Thứ ba, phần sinh nội dung phải có cơ chế kiểm soát chất lượng đầu ra đối với các sản phẩm dài như slide và giáo án. Cuối cùng, kiến trúc hệ thống phải hỗ trợ đánh giá riêng chất lượng truy hồi của *Retriever/Reranker* và chất lượng sinh của *LLM*.

0.1.2 Kiến trúc tổng thể và luồng xử lý một yêu cầu

Kiến trúc của hệ thống được tổ chức theo hai tầng *Agent* đặt trên nền *Retrieval-Augmented Generation*. Tầng điều phối tác tử đảm nhiệm việc phân tích yêu cầu,

lập kế hoạch và lựa chọn nhánh xử lý. Tầng sinh nội dung *Multi-Agent* đảm nhiệm việc tạo các sản phẩm dài như slide và giáo án. Hai tầng này cùng sử dụng một lõi truy xuất tri thức chung để bảo đảm đầu ra được grounding trên nội dung SGK.



Hình 0.1: Kiến trúc tổng thể của phương pháp đề xuất theo hướng phân tầng, trong đó lõi truy xuất tri thức là nền dùng chung cho các nhánh xử lý

Trong Hình 0.1, trục chính của vòng đời xử lý đi theo chuỗi từ yêu cầu đầu vào, phân tích ngữ cảnh và ý định, lập kế hoạch, điều phối đa tác vụ, thực thi dịch vụ nghiệp vụ, kiểm soát chất lượng đầu ra, rồi đến phản hồi và lưu trạng thái phiên. Cách biểu diễn này cho thấy bộ điều phối của hệ thống hoạt động theo một luồng xử lý chính, trong khi các năng lực truy xuất và sinh học liệu dài được gọi vào theo từng loại yêu cầu, thay vì xuất hiện như một bước tuần tự bắt buộc cho mọi truy vấn.

Ở bên phải của trục chính, khối “Truy xuất và công cụ” được đặt như một nền dùng chung cho các nhánh cần tri thức SGK. Từ khối “Dịch vụ nghiệp vụ”, hệ thống gọi xuống lõi truy xuất để lấy ngữ cảnh thông qua *Retriever*; nếu tác vụ là sinh slide hoặc giáo án thì ngữ cảnh này tiếp tục được đưa vào “Quy trình đa tác tử nội dung”. Bố cục này phản ánh quan hệ phụ thuộc giữa các thành phần: truy xuất tri thức diễn ra trước khi tầng *Multi-Agent* tổng hợp sản phẩm.

Phần cuối của sơ đồ cho thấy mọi đầu ra đều đi qua khối “Kiểm soát chất lượng đầu ra” trước khi trả về cho người dùng. Với câu trả lời ngắn, lớp này kiểm tra mức độ phù hợp và bám sát ngữ cảnh; với quiz, slide và giáo án, lớp này thực hiện hậu kiểm trước khi công bố kết quả. Hai mũi tên nét đứt cũng thể hiện rằng chat đơn giản có thể bỏ qua lõi RAG, trong khi các tác vụ cần grounding trên SGK sẽ kết

nói đến nền truy xuất chung.

0.1.3 Công nghệ nền

Về mặt công nghệ, phương pháp được hiện thực hóa trên một số thành phần nền tảng tương ứng với từng vai trò trong kiến trúc. Mô hình ngôn ngữ lớn Gemini được sử dụng như *LLM* cho các bước cần suy luận ngôn ngữ như nhận diện ý định, viết lại truy vấn và sinh nội dung. LangGraph được sử dụng cho bộ điều phối nội dung ở tầng sinh học liệu, nơi quy trình có nhiều bước trung gian và có thể tạm dừng để con người tham gia duyệt. Ở tầng truy xuất, Qdrant được sử dụng như một *Vector Database* để lưu trữ và tìm kiếm ngữ nghĩa trên tập chunk đã được lập chỉ mục.

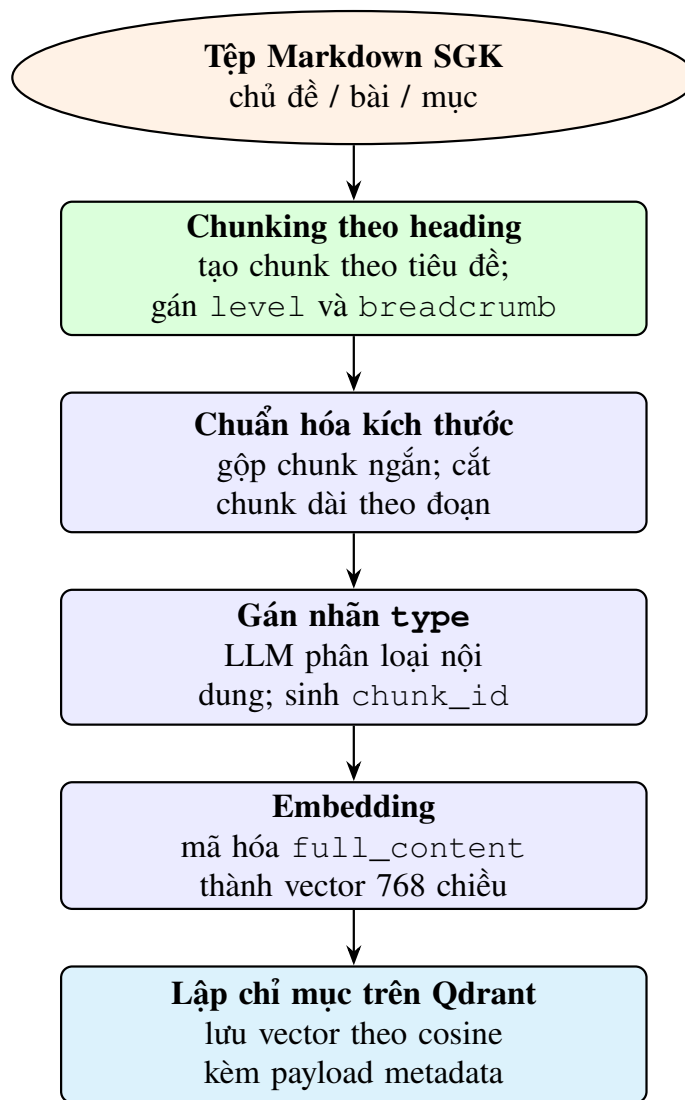
Đối với biểu diễn tri thức, hệ thống sử dụng mô hình *Embedding* tiếng Việt để chuyển mỗi khối nội dung SGK thành vector ngữ nghĩa, qua đó bảo toàn tốt hơn đặc trưng ngôn ngữ của dữ liệu trong không gian biểu diễn. Song song với đó, một thành phần tìm kiếm từ khóa kiểu BM25 được dùng để bổ sung tín hiệu lexical, sau đó kết quả được hợp nhất bằng *Reciprocal Rank Fusion*. Ở giai đoạn sau truy xuất, mô hình *Reranker* dạng cross-encoder được dùng để sắp xếp lại các ứng viên theo mức độ liên quan đối với truy vấn.

Trong phạm vi chương này, các công nghệ trên chỉ được trình bày ở mức cần thiết để làm rõ vai trò trong phương pháp đề xuất. Thông số cài đặt cụ thể của từng công nghệ sẽ được trình bày ở chương tiếp theo.

0.2 Nền tảng tri thức: tiền xử lý, chunking và lập chỉ mục

Mọi đầu ra của hệ thống đều được grounding trên nội dung sách giáo khoa, do đó chất lượng của tầng tri thức nền ảnh hưởng trực tiếp đến hiệu quả truy hồi và sinh nội dung. Phần này mô tả quy trình biến văn bản SGK thô thành một tập chunk có cấu trúc, được embedding và lập chỉ mục để phục vụ truy hồi. Hình 0.2 tóm tắt quy trình này, trong đó bước chunking theo tiêu đề (heading-based chunking) là thành phần trung tâm.

Về bản chất, tầng tri thức của hệ thống không chỉ là một kho lưu trữ dữ liệu tĩnh mà là kết quả của một pipeline tiền xử lý nhiều bước. Mỗi bước trong pipeline này tác động trực tiếp đến chất lượng truy hồi ở giai đoạn sau. Nếu ranh giới chunk không phù hợp, mô hình embedding sẽ biểu diễn sai đơn vị kiến thức; nếu metadata không đầy đủ, tầng truy hồi sẽ khó giới hạn đúng phạm vi lớp, bộ sách và bài học; nếu chỉ mục vector không đồng bộ với dữ liệu nguồn, kết quả truy xuất có thể mất tính ổn định giữa các lần chạy thực nghiệm. Vì vậy, việc mô tả rõ quy trình xây dựng tầng tri thức là cần thiết trước khi phân tích cơ chế retrieval ở mục kế tiếp.



Hình 0.2: Quy trình từ văn bản SGK đến tập chunk được lập chỉ mục, với chunking theo tiêu đề là bước trung tâm

Hình 0.2 mô tả quy trình một chiều biến tệp bài học SGK thành các chunk sẵn sàng cho truy hồi, gồm năm bước nối tiếp. Đầu vào là tệp Markdown của một bài học, trong đó cấu trúc phân cấp (chủ đề, tiêu đề bài, tiêu đề mục) đã được thể hiện qua các mốc tiêu đề. Bước trung tâm là *chunking theo heading*: một máy trạng thái quét tuần tự văn bản và chốt một chunk mới tại mỗi mốc tiêu đề, đồng thời gán trường `level` (độ sâu tiêu đề) và dựng chuỗi breadcrumb cho chunk đó. Tiếp theo, bước chuẩn hóa kích thước gộp các chunk quá ngắn và cắt các chunk quá dài để đưa độ dài về khoảng hợp lý mà không phá vỡ ranh giới ngữ nghĩa. Bước thứ tư phân loại vai trò sư phạm (`type`) của mỗi chunk bằng một mô hình ngôn ngữ nhẹ và sinh `chunk_id` bất định. Cuối cùng, mỗi chunk được embedding cùng breadcrumb thành vector 768 chiều và được lập chỉ mục trên Qdrant kèm toàn bộ payload metadata. Trong các bước này, chunking theo tiêu đề giữ vai trò quyết định vì nó định hình ranh giới và tính mạch lạc của mọi chunk ở các bước sau; ba mục

tiếp theo lần lượt trình bày chi tiết nguồn dữ liệu và metadata, cơ chế chunking, và khâu vector hóa – lưu trữ.

0.2.1 Nguồn SGK, cấu trúc phân cấp và lược đồ metadata

Dữ liệu nền của hệ thống là sách giáo khoa Tin học THPT của hai bộ Cánh Diều và Kết Nối Tri Thức, trải đều ba khối lớp 10, 11 và 12. Toàn bộ nội dung được số hóa về định dạng Markdown và tổ chức theo đúng cấu trúc phân cấp tự nhiên của sách: mỗi bộ sách (*book*) chia theo khối lớp (*grade*), mỗi lớp gồm nhiều chủ đề (*topic*), mỗi chủ đề gồm nhiều bài học (*lesson*), và mỗi bài học lại được chia thành các mục theo cấp độ tiêu đề. Cấu trúc thư mục phản ánh trực tiếp phân cấp này, trong đó mỗi tệp Markdown tương ứng với một bài học và tệp `README.md` ở mỗi thư mục chủ đề lưu mã cùng tên chủ đề.

Sau bước xử lý, tập tri thức thu được gồm 2.348 chunk, bao phủ 36 chủ đề và 186 bài học khác nhau. Bảng 1 trình bày phân bố số lượng chunk theo bộ sách và khối lớp.

Bộ sách	Lớp 10	Lớp 11	Lớp 12	Tổng
Cánh Diều (CD)	401	439	364	1.204
Kết Nối Tri Thức (KNTT)	401	350	393	1.144
Tổng	802	789	757	2.348

Bảng 1: Phân bố số lượng chunk theo bộ sách và khối lớp

Số liệu trong Bảng 1 cho thấy dữ liệu tương đối cân bằng giữa hai bộ sách, với 1.204 chunk thuộc bộ Cánh Diều và 1.144 chunk thuộc bộ Kết Nối Tri Thức. Phân bố theo khối lớp cũng không có chênh lệch lớn, lần lượt là 802 chunk ở lớp 10, 789 chunk ở lớp 11 và 757 chunk ở lớp 12. Phân bố này giúp giảm nguy cơ tăng truy hồi thiên lệch về một bộ sách hay một khối lớp cụ thể.

Mỗi chunk được gắn một lược đồ metadata thống nhất nhằm phục vụ lọc phạm vi khi truy hồi và truy vết nguồn khi sinh nội dung. Bảng 2 mô tả các trường chính của lược đồ này.

Bảng 2 cho thấy mỗi chunk không chỉ lưu nội dung văn bản mà còn mang theo các trường metadata về phạm vi học liệu, vị trí phân cấp và nguồn gốc dữ liệu. Tập trường này là cơ sở để tăng truy hồi thực hiện lọc theo bộ sách, lớp, chủ đề và bài học, đồng thời hỗ trợ truy vết nguồn ở các bước sinh nội dung và đánh giá.

Trường `type` phân loại mỗi chunk theo vai trò sư phạm thay vì chỉ theo hình thức văn bản, gồm sáu nhãn: *theory* (lý thuyết, khái niệm, ví dụ), *exercise* (luyện tập, bài tập), *objective* (mục tiêu bài học), *application* (vận dụng), *summary* (tóm tắt) và *note* (lưu ý). Bảng 3 trình bày phân bố số lượng chunk theo từng nhãn.

Trường	Kiểu	Ý nghĩa
chunk_id	uuid	Định danh duy nhất, sinh tất định
book	str	Bộ sách (CD / KNTT)
grade	str	Khối lớp (10 / 11 / 12)
topic, topic_name	str	Mã và tên chủ đề
lesson, lesson_name	str	Mã và tên bài học
section_title	str	Tiêu đề mục chứa chunk
level	int	Độ sâu tiêu đề (1–6)
type	str	Loại nội dung sư phạm
breadcrumb	str	Đường dẫn ngữ cảnh: chủ đề > bài > mục
content	str	Nội dung văn bản của chunk
full_content	str	Ghép breadcrumb và content để vector hóa
source_path	str	Đường dẫn tệp nguồn

Bảng 2: Lược đồ metadata của một chunk

Loại (type)	Vai trò	Số lượng	Tỷ lệ
<i>theory</i>	Lý thuyết, khái niệm, ví dụ	1.682	71,6%
<i>exercise</i>	Luyện tập, bài tập	250	10,6%
<i>objective</i>	Mục tiêu bài học	193	8,2%
<i>application</i>	Vận dụng	122	5,2%
<i>summary</i>	Tóm tắt	85	3,6%
<i>note</i>	Lưu ý	16	0,7%
Tổng		2.348	100%

Bảng 3: Phân bố số lượng chunk theo loại nội dung sư phạm (type)

Số liệu trong Bảng 3 cho thấy phần lớn nội dung thuộc nhãn *theory*, chiếm khoảng 71,6% tổng số chunk. Phân bố này phản ánh đặc thù của SGK, trong đó phần trình bày khái niệm, giải thích và ví dụ chiếm ưu thế so với các thành phần như bài tập, mục tiêu hay tóm tắt. Nhãn loại nội dung này tạo cơ sở cho việc khai thác chọn lọc ở các tác vụ phía sau, chẳng hạn ưu tiên chunk *objective* và *summary* khi dựng dàn ý bài giảng, hoặc chunk *exercise* khi sinh câu hỏi luyện tập.

0.2.2 Chunking theo tiêu đề

Khác với cách chia chunk theo cửa sổ ký tự cố định, chiến lược **chunking theo tiêu đề (heading-based chunking)** được sử dụng trong đồ án xác định ranh giới chunk trực tiếp từ cấu trúc tiêu đề Markdown của SGK. Theo đó, mỗi chunk tương ứng với một đơn vị nội dung mạch lạc về mặt sư phạm. Cụ thể, bộ chunking hoạt động như một máy trạng thái quét tuần tự từng dòng của tệp bài học; mỗi khi gặp một mốc phân cấp — dòng chủ đề (“CHỦ ĐỀ ...”), tiêu đề bài (“# Bài X: ...”) hay tiêu đề mục (“## ...”, “### ...”) — hệ thống chốt chunk đang tích lũy và mở một chunk mới gắn với mốc đó. Cách chia này bảo đảm mỗi chunk nằm trọn trong một mục của một bài, không bị cắt ngang giữa câu hay giữa các ý.

Để làm rõ cơ chế này, có thể xét một tài liệu giả lập gồm ba tầng tiêu đề như sau:

CHU DE 2. Mạng máy tính và Internet

Bài 3. Giao tiếp trên mạng

Doan mo dau cua bai hoc.

1. Địa chỉ IP

Địa chỉ IP dùng để định danh thiết bị trên mạng.

Ví dụ

Máy tính A có địa chỉ 192.168.1.10.

2. Tên miền

Tên miền giúp người dùng dễ nhớ địa chỉ tại nguyên hơn.

Với đầu vào trên, hệ thống không tạo một chunk duy nhất cho toàn bộ bài học mà chốt chunk tại từng mốc tiêu đề. Bảng 4 minh họa kết quả phân đoạn tương ứng.

Chunk giả lập	Level	Breadcrumb	Nội dung chính
c_1	1	Chủ đề 2 > Bài 3	Đoạn mở đầu của bài học
c_2	2	Chủ đề 2 > Bài 3 > 1. Địa chỉ IP	Khái niệm địa chỉ IP
c_3	3	Chủ đề 2 > Bài 3 > 1. Địa chỉ IP > Ví dụ	Ví dụ về địa chỉ IP cụ thể
c_4	2	Chủ đề 2 > Bài 3 > 2. Tên miền	Khái niệm tên miền

Bảng 4: Ví dụ minh họa cách chunking theo tiêu đề trên một tài liệu ba tầng heading

Bảng 4 cho thấy ranh giới chunk được quyết định bởi tiêu đề gần nhất đang có hiệu lực. Đoạn mở đầu ngay sau tiêu đề bài được gán vào chunk mức 1. Khi gặp mục “1. Địa chỉ IP”, hệ thống tạo một chunk mới ở mức 2. Phần “Ví dụ” bên trong mục này tiếp tục được tách thành một chunk mức 3, do nó được mở bởi một tiêu đề con riêng. Tương tự, khi gặp mục “2. Tên miền”, hệ thống chốt chunk trước đó và mở ra một chunk mới. Nhờ cách chia này, mỗi chunk giữ được cả nội dung trực tiếp lẫn vị trí phân cấp trong bài học.

Mỗi chunk được gán trường `level` bằng đúng độ sâu tiêu đề trong Markdown: tiêu đề bài tương ứng `level = 1`, các mục chính `level = 2`, các mục con `level ≥ 3`. Phân bố thực tế cho thấy nội dung tập trung chủ yếu ở `level 2` (1.229 chunk) và `level 3` (921 chunk). Song song, mỗi chunk mang một chuỗi

breadcrumb dạng “tên chủ đề > tên bài > tiêu đề mục” làm ngữ cảnh phân cấp gọn nhẹ; breadcrumb được ghép vào trước nội dung khi embedding và khi xếp hạng lại bằng *Reranker*, qua đó hỗ trợ phân biệt các chunk có câu chữ tương tự nhưng thuộc bài khác nhau.

Vì ranh giới chunk bám theo tiêu đề nên kích thước chunk phụ thuộc độ dài tự nhiên của từng mục. Để xử lý hai trường hợp cực đoan, một bước chuẩn hóa kích thước nhẹ được áp dụng sau khi chia: các chunk quá ngắn (dưới 100 ký tự) được gộp vào chunk liền trước nếu cùng bài và cùng loại nội dung, còn các chunk quá dài (trên 2.000 ký tự) được cắt theo ranh giới đoạn văn. Các ngưỡng này chỉ tác động tới một phần nhỏ tập chunk và không làm thay đổi bản chất chia theo tiêu đề; độ dài chunk thực tế dao động quanh trung bình 651 ký tự. Cuối cùng, nhãn `type` được tinh chỉnh bằng một mô hình ngôn ngữ nhẹ (Gemini 2.5 Flash-Lite) trên cơ sở nhãn heuristic ban đầu, có cơ chế đệm (cache) để bảo đảm tính tái lập; đồng thời mỗi chunk nhận một `chunk_id` sinh tất định bằng hàm băm trên (bộ sách, lớp, chủ đề, bài, breadcrumb, thứ tự, dấu vân nội dung), giúp việc tái dựng tập chunk có tính idempotent.

0.2.3 Biểu diễn vector tiếng Việt và lưu trữ Qdrant

Để hỗ trợ truy hồi ngữ nghĩa, mỗi chunk được chuyển thành một vector số thực bằng mô hình embedding tiếng Việt `dangvantuan/vietnamese-document-embedding`, sinh vector 768 chiều. Văn bản đưa vào mô hình là trường `full_content` (breadcrumb ghép với nội dung) thay vì chỉ riêng nội dung, để ngữ cảnh phân cấp được phản ánh trong không gian vector. Sau khi mã hóa, các vector được chuẩn hóa L_2 nhằm bảo đảm độ đo cosine quy về tích vô hướng; quá trình này có kiểm tra và xử lý các vector suy biến (chuẩn gần không hoặc giá trị không hữu hạn) để duy trì tính ổn định của tập chỉ mục.

Tập vector cùng metadata được lưu vào Qdrant dưới một collection tên `educational_chatbot`, cấu hình không gian 768 chiều với độ đo khoảng cách cosine. Mỗi điểm (point) trong Qdrant nhận `chunk_id` làm định danh, mang vector ngữ nghĩa và toàn bộ payload metadata ở Bảng 2; nhờ đó hệ thống có thể vừa tìm kiếm theo độ tương đồng vừa lọc cứng theo bộ sách, lớp, chủ đề trong cùng một truy vấn. Quá trình nạp dữ liệu được thực hiện theo lô và có kiểm tra tính nhất quán giữa số chunk và số vector. Ngoài ra, một dấu vân (fingerprint) trên nguồn embedding được lưu kèm để phát hiện khi dữ liệu chunk thay đổi và chỉ tính lại vector khi cần. Thiết kế lưu trữ này cho phép tăng truy hồi nạp dữ liệu từ kho Qdrant hoặc từ tệp vector cục bộ trong các kịch bản triển khai và thực nghiệm.

0.3 Cơ chế truy hồi ngữ cảnh (RAG core)

0.3.1 Truy hồi lai BM25 và dense, hợp nhất bằng Reciprocal Rank Fusion

Lỗi truy hồi của hệ thống được xây dựng theo hướng truy hồi lai, kết hợp giữa tìm kiếm từ khóa (*keyword search*) và tìm kiếm ngữ nghĩa theo vector (*vector search*). Trong đó, *keyword search* phù hợp với các truy vấn có mức trùng khớp bề mặt cao với SGK, còn *vector search* phù hợp với các truy vấn diễn đạt lại hoặc dùng từ đồng nghĩa.

Với *keyword search*, hệ thống sử dụng BM25. Xuất phát từ TF-IDF, trong đó tần suất từ được ký hiệu bởi Công thức 1:

$$TF(t, d) = f(t, d), \quad (1)$$

và tần suất nghịch đảo tài liệu được tính theo Công thức 2:

$$IDF(t) = \log \frac{N - n(t) + 0.5}{n(t) + 0.5}, \quad (2)$$

Trong đó:

- t là một term trong truy vấn.
- N là số tài liệu trong tập chỉ mục.
- $n(t)$ là số tài liệu chứa từ t .

Trên cơ sở đó, điểm BM25 của truy vấn q đối với chunk d được tính như sau:

$$BM25(q, d) = \sum_{t \in q} IDF(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)}. \quad (3)$$

Trong đó:

- q là truy vấn.
- t là một term thuộc truy vấn q .
- $f(t, d)$ là số lần xuất hiện của từ t trong chunk d .
- $|d|$ là độ dài của chunk d .
- $avgdl$ là độ dài chunk trung bình của toàn bộ tập chỉ mục.
- k_1 và b là các siêu tham số điều chỉnh ảnh hưởng của tần suất từ và độ dài tài liệu.

Với *vector search*, mỗi truy vấn q và mỗi chunk d được ánh xạ thành embedding $\mathbf{e}_q$ và $\mathbf{e}_d$. Độ tương đồng được tính bằng cosine similarity theo Công thức 4:

$$\text{cos_sim}(\mathbf{e}_q, \mathbf{e}_d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_d\|_2}. \quad (4)$$

Trong đó:

- $\mathbf{e}_q$ là embedding của truy vấn.
- $\mathbf{e}_d$ là embedding của chunk.
- $\mathbf{e}_q \cdot \mathbf{e}_d$ là tích vô hướng giữa hai vector.
- $\|\cdot\|_2$ là chuẩn Euclid.

Do các vector được chuẩn hóa L_2 trước khi lập chỉ mục, độ đo này quy về so sánh hướng trong không gian vector và được dùng trực tiếp trong truy vấn Qdrant.

Hai danh sách kết quả từ BM25 và *vector search* được hợp nhất bằng *Reciprocal Rank Fusion* (RRF), thay vì trộn trực tiếp điểm số. Với một chunk d , điểm RRF được xác định theo Công thức 5:

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)}, \quad (5)$$

Trong đó:

- $\mathcal{R}$ là tập các danh sách xếp hạng thành phần.
- $\text{rank}_r(d)$ là thứ hạng của chunk d trong danh sách r .
- k là hằng số làm tròn.

Cách hợp nhất này tránh phụ thuộc vào khác biệt thang điểm giữa BM25 và cosine similarity, đồng thời giữ lại các chunk có thứ hạng cao ở ít nhất một kênh truy hồi. Kết quả của bước này là một tập candidate ban đầu để chuyển sang bước reranking.

0.3.2 Xếp hạng lại bằng cross-encoder

Sau bước truy hồi lại, hệ thống áp dụng một bước *reranking* bằng cross-encoder để tái xếp hạng tập candidate. Mục tiêu của bước này là thay thế thứ tự dựa trên mức tương đồng sơ cấp bằng thứ tự gần hơn với mức độ liên quan thực sự giữa truy vấn và chunk.

Reranker nhận đầu vào là cặp (*query*, *context*) và sinh ra một điểm số liên quan

$$s_{\text{rerank}} = f_{\theta}(q, d), \quad (6)$$

Trong đó:

- q là truy vấn.
- d là chunk ứng viên.
- θ là tập tham số của mô hình.
- f_{θ} là hàm chấm điểm của cross-encoder.

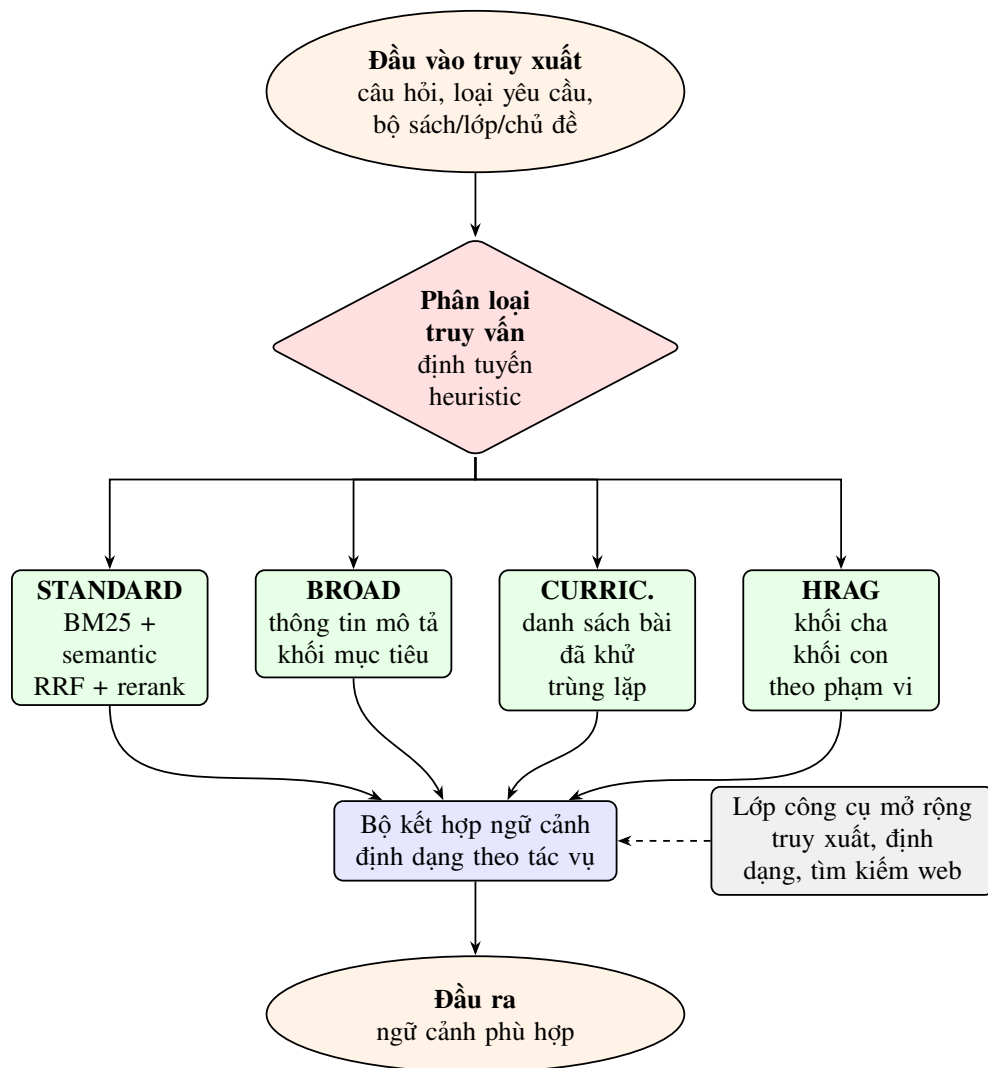
Khác với *vector search*, mô hình này mã hóa đồng thời cả truy vấn và chunk, do đó có thể khai thác tương tác token-level giữa hai vế. Trong triển khai hiện tại, phần văn bản đưa vào *Reranker* không chỉ gồm *content* mà còn ghép thêm *breadcrumb*, nhằm cung cấp thêm ngữ cảnh phân cấp của chunk trong SGK.

Sau khi tính lại điểm số, hệ thống chọn top- N chunk để xây dựng ngữ cảnh cuối. Giá trị N được đặt theo loại tác vụ: với hỏi đáp ngắn, N nhỏ hơn để hạn chế nhiễu; với slide hoặc giáo án, N lớn hơn để tăng độ bao phủ. Vì vậy, reranking vừa là bước cải thiện thứ hạng, vừa là bước chọn lọc ngữ cảnh đầu vào cho tầng generation.

0.3.3 Truy hồi thích ứng (Adaptive RAG)

Không phải mọi truy vấn trong hệ thống đều cần cùng một cách truy hồi. Câu hỏi định nghĩa ngắn, yêu cầu liệt kê bài học, truy vấn tổng quan theo chủ đề và yêu cầu sinh slide từ một bài học có phạm vi ngữ cảnh khác nhau. Vì vậy, hệ thống sử dụng cơ chế *Adaptive RAG* để chọn strategy truy hồi phù hợp theo loại truy vấn và thông tin phạm vi hiện có.

Về mặt lý thuyết, cơ chế này có hai thành phần chính. Thành phần thứ nhất là bước phân loại truy vấn, nhằm xác định loại nhu cầu truy hồi trước khi chọn strategy. Thành phần thứ hai là bước ràng buộc phạm vi (*scope filtering*) theo bộ sách, lớp hoặc chủ đề nếu các metadata này đã được nhận diện ở tầng điều phối. Nhờ đó, không gian tìm kiếm được thu hẹp trước khi thực hiện BM25 hoặc *vector search*. Trong trường hợp strategy đang dùng không trả về đủ ngữ cảnh, hệ thống có thể fallback về cấu hình mặc định để bảo đảm độ phủ.



Hình 0.3: Cơ chế truy hồi thích ứng của hệ thống với bộ phân loại truy vấn và các chiến lược truy xuất tương ứng

Trong Hình 0.3, bộ phân loại truy vấn được đặt ở trung tâm vì đây là thành phần quyết định strategy truy hồi sẽ được áp dụng trước. Bốn strategy ở tầng dưới tương ứng với bốn kiểu nhu cầu truy hồi khác nhau, trong khi khối “Bộ kết hợp ngữ cảnh” chuẩn hóa đầu ra của từng strategy về một cấu trúc chung cho tầng trên sử dụng. Trong cấu trúc này, *STANDARD* là strategy mặc định gồm BM25, *vector search*, RRF và reranking. *BROAD* ưu tiên các chunk mô tả ở mức khái quát. *CURRICULUM* phục vụ các truy vấn liệt kê bài học hoặc cấu trúc chương trình, trong đó đầu ra cần ở mức danh sách bài thay vì đoạn nội dung chi tiết. *HRAG* kết hợp các chunk mức cao và mức chi tiết trong cùng phạm vi bài học. Sau khi truy hồi, khối “Bộ kết hợp ngữ cảnh” thực hiện chuẩn hóa đầu ra để các strategy khác nhau có thể cung cấp ngữ cảnh theo cùng một cấu trúc cho tầng trên sử dụng.

0.4 Tầng điều phối tác tử

0.4.1 Sự cần thiết của tầng điều phối tác tử

Nếu hệ thống chỉ phục vụ một tác vụ hỏi đáp đơn giản, pipeline tuyến tính gồm phân tích truy vấn, truy xuất và sinh câu trả lời là đủ. Tuy nhiên, bài toán của đề án bao gồm đồng thời nhiều nhóm yêu cầu khác nhau, từ hỏi đáp khái niệm, giải thích bài học, sinh slide đến soạn giáo án. Các yêu cầu này khác nhau không chỉ ở định dạng đầu ra mà còn ở cách khai thác tri thức, mức độ cần hội thoại nhiều lượt và số bước xử lý trung gian. Do đó, việc sử dụng một pipeline cố định cho mọi truy vấn làm giảm khả năng thích ứng của hệ thống.

Tầng điều phối tác tử được đưa vào để tách riêng trách nhiệm *hiểu yêu cầu, quyết định nhánh xử lý* và *thực thi các hành động cần thiết*. Trong kiến trúc này, bộ điều phối không được thiết kế như một *Agent* tự trị bao trùm toàn bộ hệ thống. Thay vào đó, nó là một lớp điều khiển mỏng, giữ trạng thái yêu cầu, gọi *LLM* ở những vị trí cần suy luận ngôn ngữ như nhận diện ý định hoặc viết lại truy vấn, và dùng logic xác định cho các bước lập kế hoạch, định tuyến và quản lý phiên. Cách tổ chức này giúp giảm số lượt gọi mô hình, hạn chế độ trễ và làm rõ ranh giới giữa quyết định dựa trên ngôn ngữ với quyết định có thể diễn đạt bằng luật.

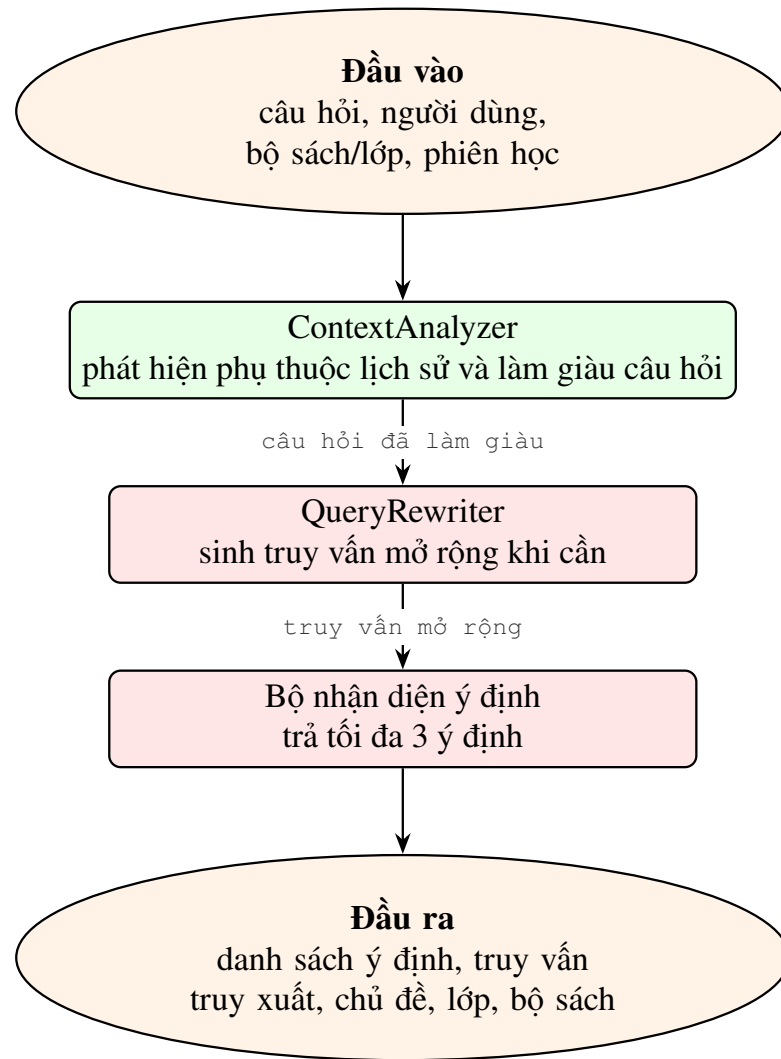
Vai trò thứ hai của tầng điều phối là hỗ trợ yêu cầu đa ý định. Trong thực tế sử dụng, một câu hỏi có thể đồng thời chứa nhiều hành động, chẳng hạn vừa yêu cầu giải thích khái niệm, vừa yêu cầu tạo bài tập hoặc sinh dàn ý bài giảng. Khi đó, hệ thống cần phân rã yêu cầu đầu vào thành một danh sách hành động có thứ tự, tránh xử lý trùng lặp và duy trì được trạng thái xuyên suốt phiên làm việc. Đây chính là lý do phần điều phối được mô hình hóa thành một tầng riêng, thay vì gắn trực tiếp vào từng dịch vụ nghiệp vụ.

0.4.2 Định tuyến ý định bằng mô hình ngôn ngữ lớn

Điểm vào đầu tiên của tầng điều phối là bài toán nhận diện ý định (*intent routing*). Với cùng một câu hỏi bề mặt, hệ thống cần xác định truy vấn đang hướng tới hỏi đáp kiến thức, sinh học liệu dài, hay một thao tác phụ trợ khác như thống kê tiến độ hoặc ôn câu sai. Ngoài loại ý định, hệ thống còn cần trích xuất các trường phạm vi như bộ sách, lớp, chủ đề và bài học khi thông tin này xuất hiện trong câu hỏi hoặc có thể suy ra từ ngữ cảnh phiên.

Trong phương pháp đề xuất, bước định tuyến ý định được giao cho *LLM* thay vì heuristic thuần túy. Lý do là cùng một ý định có thể được diễn đạt theo nhiều cách khác nhau, đặc biệt trong ngôn ngữ học sinh. Bộ định tuyến cần xử lý được các câu truy vấn gián tiếp, câu truy vấn ghép và cả các trường hợp một câu hỏi ngắn nhưng phụ thuộc mạnh vào lịch sử hội thoại. Đầu ra của bước này không phải chỉ là một

nhân duy nhất, mà là một danh sách tối đa ba ý định có thứ tự ưu tiên, đi kèm thông tin phạm vi phục vụ các tầng sau.



Hình 0.4: Khối phân tích ngữ cảnh và định tuyến ý định trong tầng điều phối

Hình 0.4 mô tả chuỗi xử lý từ câu hỏi gốc đến đầu ra định tuyến. Đầu vào ban đầu gồm câu hỏi hiện tại, thông tin người dùng, bộ sách hoặc lớp được chọn trên giao diện và trạng thái phiên học. Thành phần *ContextAnalyzer* đứng trước cùng vì đây là bước quyết định câu hỏi hiện tại có phụ thuộc vào lịch sử hội thoại hay không. Nếu phụ thuộc, hệ thống lấy các lượt hội thoại gần và liên quan để mở rộng câu hỏi hiện tại trước khi chuyển sang bước sau.

Sau bước phân tích ngữ cảnh, hệ thống chỉ gọi *QueryRewriter* khi truy vấn cần được diễn đạt lại để hỗ trợ *retrieval*. Thành phần này không thay thế câu hỏi gốc, mà tạo ra một hoặc nhiều truy vấn mở rộng nhằm tăng khả năng bao phủ của tầng truy hồi trong các trường hợp câu hỏi quá ngắn, thiếu chủ ngữ hoặc dùng diễn đạt không trùng với SGK. Cuối cùng, bộ nhận diện ý định dùng *LLM* để phân tích yêu cầu đã được làm giàu và trả về danh sách ý định, truy vấn phục vụ truy xuất và các

trường phạm vi như chủ đề, lớp và bộ sách. Đầu ra này là đầu vào trực tiếp cho tầng lập kế hoạch.

0.4.3 Làm giàu ngữ cảnh hội thoại và viết lại truy vấn

Làm giàu ngữ cảnh hội thoại là bước cần thiết vì nhiều truy vấn trong thực tế không tự mang đủ thông tin để tăng truy hồi hoạt động chính xác. Các câu như “giải thích kỹ hơn”, “bài này khác gì bài trước” hoặc “làm cho em vài câu hỏi” chỉ có thể được hiểu đúng nếu hệ thống nối chúng với lượt hội thoại trước đó. Do vậy, trước khi viết lại truy vấn, hệ thống trước hết phải xác định liệu câu hỏi hiện tại có phụ thuộc vào lịch sử hay không.

Trong hệ thống, *ContextAnalyzer* được hiện thực bằng logic xác định. Thành phần này quan sát câu hỏi hiện tại, trạng thái phiên và một số tín hiệu bề mặt như đại từ hồi chỉ, cấu trúc câu rút gọn hoặc việc người dùng tiếp tục một chủ đề đã mở trước đó. Khi phát hiện phụ thuộc ngữ cảnh, bộ phân tích trích xuất một cửa sổ hội thoại ngắn, ưu tiên các lượt gần nhất và các lượt có cùng phạm vi bài học. Kết quả của bước này là một phiên bản câu hỏi đã được làm rõ, đủ để tầng dưới xử lý mà không cần mang theo toàn bộ lịch sử hội thoại.

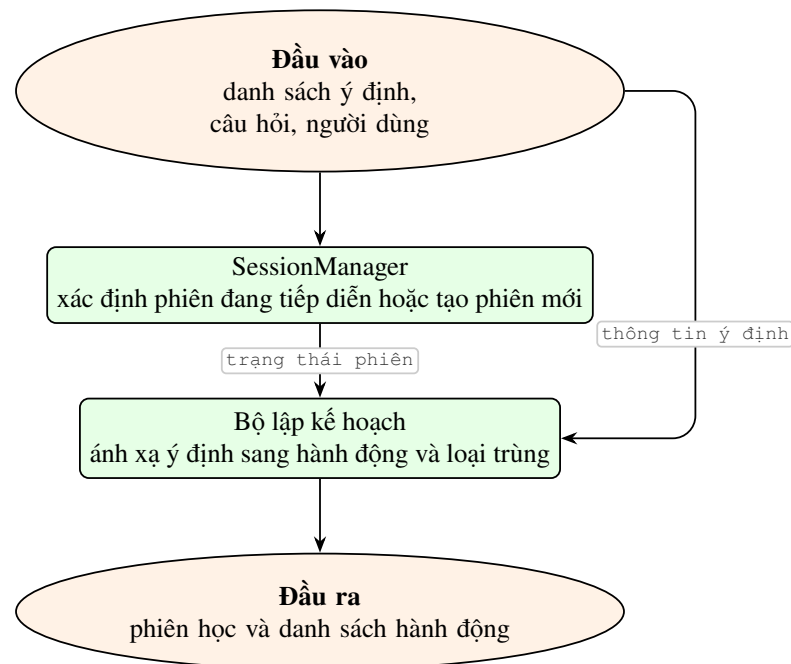
QueryRewriter đóng vai trò chuyển câu hỏi đã làm giàu thành dạng phù hợp hơn cho *retrieval*. Với truy vấn hỏi đáp, bộ viết lại có thể sinh thêm các biến thể gần nghĩa để tăng độ phủ của *keyword search* và *vector search*. Với truy vấn sinh học liệu dài, bộ viết lại ưu tiên làm rõ phạm vi bài học, chủ đề và kiểu đầu ra cần sinh. Việc tách bước này khỏi *IntentRouter* giúp hệ thống kiểm soát được hai bài toán khác nhau: một bài toán là hiểu người dùng muốn làm gì, bài toán còn lại là diễn đạt yêu cầu đó theo cách thuận lợi cho tầng truy hồi.

Trong thiết kế hiện tại, ngữ cảnh hội thoại sau khi được phân tích không được đưa thẳng vào toàn bộ pipeline ở mọi bước. Thay vào đó, nó được nén thành các trường có cấu trúc, gồm câu hỏi đã làm rõ, truy vấn mở rộng, phạm vi bộ sách/lớp/chủ đề và cờ đánh dấu mức độ phụ thuộc lịch sử. Cách chuẩn hóa này giúp các thành phần lập kế hoạch và truy hồi dùng lại cùng một biểu diễn đầu vào, thay vì mỗi thành phần tự diễn giải lịch sử theo cách riêng.

0.4.4 Lập kế hoạch hành động và thực thi đa hành động

Sau khi có danh sách ý định, hệ thống chuyển sang bước lập kế hoạch hành động (*action planning*). Mục tiêu của bước này là ánh xạ từng ý định sang một hành động nghiệp vụ cụ thể, đồng thời quyết định thứ tự thực thi khi trong cùng một yêu cầu xuất hiện nhiều ý định. Khác với bước nhận diện ý định, bài toán lập kế hoạch ở đây được giải quyết bằng logic xác định thay vì *LLM*. Lý do là tập hành động của hệ thống là hữu hạn và đã biết trước, bao gồm hỏi đáp, giải thích khái

niệm, sinh slide, sinh giáo án, sinh quiz, chấm đáp án, ôn câu sai và một số thao tác hỗ trợ khác.

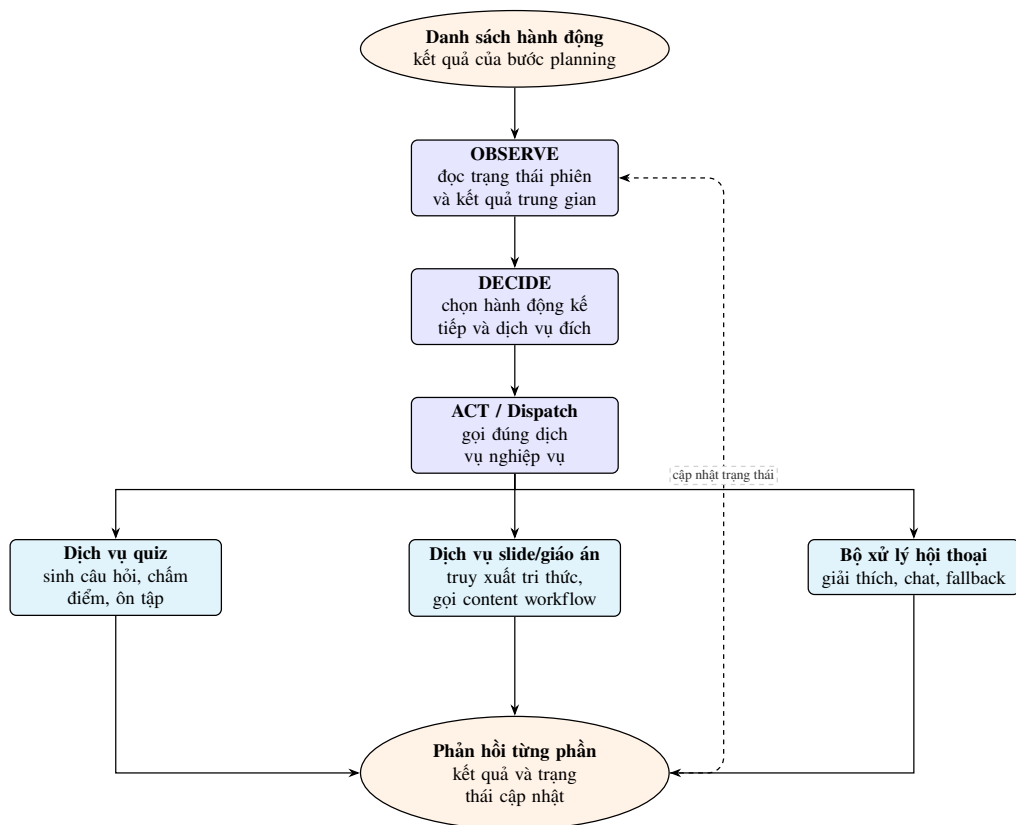


Hình 0.5: Cơ chế lập kế hoạch hành động ở tầng điều phối

Hình 0.5 cho thấy bước lập kế hoạch gồm hai khối chính. Trước hết, *SessionManager* xác định phiên học hiện tại của người dùng, từ đó quyết định nên tiếp tục luồng đang có hay khởi tạo phiên mới. Tiếp theo, bộ lập kế hoạch nhận đồng thời thông tin ý định và trạng thái phiên để sinh ra một danh sách hành động đã được loại trùng. Chẳng hạn, nếu nhiều ý định liên tiếp cùng dẫn tới một hành động hỏi đáp trong cùng một phạm vi bài học, hệ thống không tạo nhiều bước xử lý trùng nhau mà gộp về một hành động duy nhất.

Sau bước lập kế hoạch, danh sách hành động được thực thi theo một vòng lặp điều phối đa hành động. Về bản chất, đây là một chu trình *OBSERVE-DECIDE-ACT*: hệ thống quan sát trạng thái hiện tại của phiên và kết quả của hành động trước đó, quyết định hành động kế tiếp cần chạy, rồi gọi đúng dịch vụ nghiệp vụ tương ứng. Cấu trúc này cho phép hệ thống trả dần kết quả của từng hành động, đồng thời cập nhật trạng thái trung gian mà không cần chờ toàn bộ yêu cầu phức hợp kết thúc mới sinh phản hồi.

Hình 0.6 làm rõ bước thực thi sau lập kế hoạch. Khối *OBSERVE* đọc trạng thái phiên và các kết quả trung gian đã có. Khối *DECIDE* chọn hành động kế tiếp cần chạy dựa trên danh sách hành động còn lại. Khối *ACT* tương ứng với bộ định tuyến thực thi (*dispatcher*), chịu trách nhiệm gọi đúng dịch vụ nghiệp vụ. Với các hành động gắn với quiz, luồng được chuyển sang dịch vụ quiz. Với các hành động cần



Hình 0.6: Vòng thực thi đa hành động trong tầng điều phối theo chu trình quan sát – quyết định – hành động

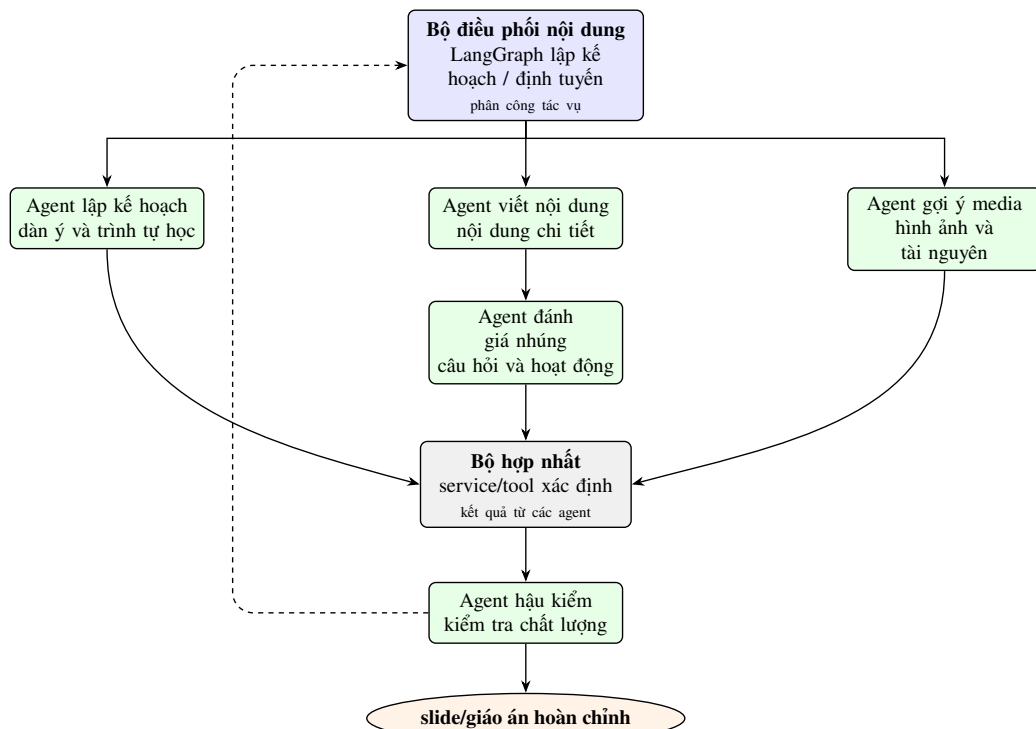
sinh học liệu dài, luồng được chuyển sang dịch vụ slide/giáo án để tiếp tục gọi nhánh *Multi-Agent*. Với các hành động giải thích hoặc hội thoại ngắn, hệ thống dùng bộ xử lý hội thoại chuyên biệt. Kết quả sau mỗi vòng được đưa trở lại trạng thái phiên để phục vụ vòng kế tiếp.

Thiết kế này cho phép bộ điều phối giữ vai trò trung tâm của phương pháp đề xuất mà không làm nó trở thành một khối suy luận quá lớn. *LLM* chỉ xuất hiện ở những bước cần hiểu ngôn ngữ hoặc sinh biến thể truy vấn; còn việc quản lý trạng thái, lập kế hoạch và điều phối hành động vẫn được thực hiện bằng cơ chế xác định. Nhờ đó, hệ thống duy trì được tính giải thích được của luồng xử lý, đồng thời vẫn đủ linh hoạt để hỗ trợ các yêu cầu phức hợp trong cùng một phiên tương tác.

0.5 Tầng sinh nội dung đa tác tử

0.5.1 Mô hình Supervisor–Specialist và lý do lựa chọn

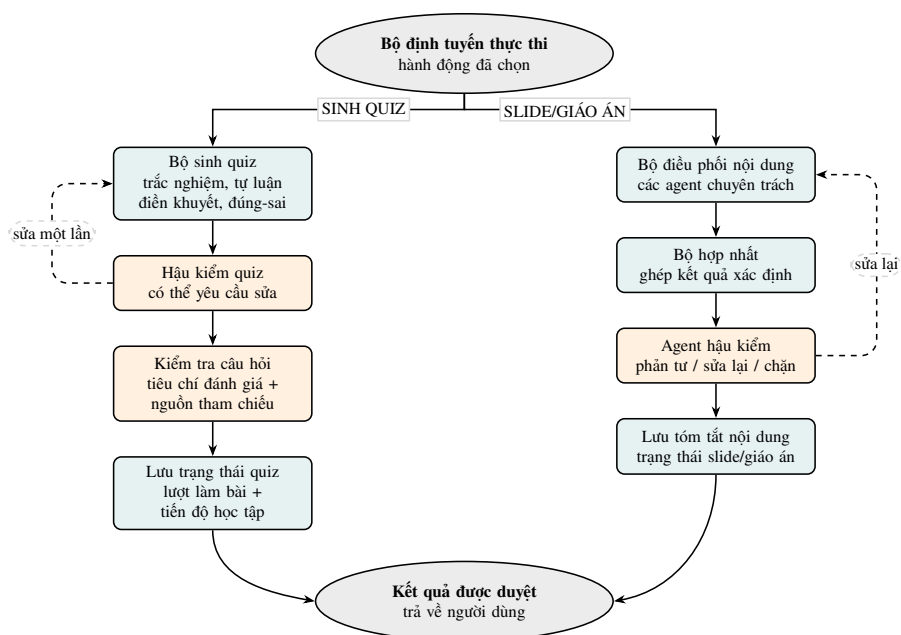
0.5.2 Giao thức giao tiếp và cơ chế điều phối giữa các tác tử



Hình 0.7: Kiến trúc đa tác tử cho bài toán sinh slide và giáo án

0.5.3 Các tác tử chuyên biệt trong pipeline sinh nội dung

0.5.4 Tổng hợp kết quả, kiểm soát chất lượng và con người trong vòng lặp



Hình 0.8: Hai nhánh sinh nội dung và cơ chế hậu kiểm chất lượng của hệ thống

0.5.5 Tái sử dụng kiến trúc cho các tác vụ sinh nội dung

0.6 Phương pháp đánh giá

0.6.1 Đánh giá truy hồi

0.6.2 Đánh giá chất lượng sinh

0.7 Kết chương